

PLANUMO — TRAVEL PACKAGE MARKETPLACE

A PROJECT REPORT

Submitted by
2116230701201 – MYMOON BEEVI B
2116230701520 – RAGHAVAN S



April,2026

BONAFIDE CERTIFICATE

Certified that this project report PLANIMO — TRAVEL PACKAGE MARKETPLACE is the bonafide work of Mymoon Beevi B(230701201), Raghavan S(230701520) who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

- 1. INTRODUCTION
- 2. SCOPE OF THE WORK
- 3. PROJECT TIMELINE
- 3.5 AIM AND OBJECTIVES OF THE PROJECT
- 4. LITERATURE SURVEY
 - 4.1 Mobile Frameworks
 - 4.2 State Management and Routing
 - 4.3 UI Standards
 - 4.4 Microservices and Concurrency
- 5. SYSTEM DESIGN
 - 5.1 Architecture Overview
 - 5.2 Transactional Outbox Pattern
 - 5.3 Data Flow and Service Isolation
- 6. MODULE DESCRIPTION
- 6.5 KEY BENEFITS OF THE PROPOSED SYSTEM
- 7. ARCHITECTURE DIAGRAM
- 8. USE CASE DIAGRAM
- 9. HARDWARE SPECIFICATIONS
- 10. SOFTWARE SPECIFICATIONS
- 11. TESTING AND VALIDATION
- 13. FUTURE ENHANCEMENTS
- 15. CONCLUSION
- 16. REFERENCES

ABSTRACT

Planymo is a sophisticated, full-stack travel-package marketplace engineered to bridge the digital divide between travelers and vendors. By integrating a Flutter-based mobile interface with a robust microservices backend, the platform enables high-concurrency booking and real-time synchronization. The architecture prioritizes data integrity through the Transactional Outbox Pattern and ensures system resilience via Apache Kafka, providing a comprehensive solution for the modern tourism industry.

1. INTRODUCTION

The global tourism industry demands seamless digital experiences, yet current platforms often struggle with fragmented inventory and lack of verified trust. Planymo addresses these challenges by providing a unified ecosystem for curated travel packages. Our solution leverages an event-driven "central nervous system" architecture, allowing independent services like booking, reviews, and notifications to communicate asynchronously. This design choice ensures that even during peak seasonal traffic, the core booking flow remains uninterrupted and highly responsive.

2. SCOPE OF THE WORK

The project scope is defined by the technical orchestration of three primary tiers: mobile clients, vendor dashboards, and a distributed backend infrastructure. Each component is designed to solve specific operational challenges in the travel marketplace.

- **Flutter Mobile Application:** Targeting Android and iOS from a single codebase.
- **Authentication:** Secure JWT-based access with silent refresh mechanisms.
- **Discovery Engine:** Real-time search and package categorization.
- **Support Panel:** A dedicated dashboard for managing tickets, viewing system health, and monitoring payment status.
- **Backend Infrastructure:** Microservices implemented in Django REST Framework.

3. PROJECT TIMELINE

The development lifecycle followed an iterative Agile methodology, beginning with intensive requirement gathering and technical feasibility studies in mid-2025. This phase ensured that the subsequent design and implementation phases were grounded in real-world user needs and technical constraints.

Task	Start Date	End Date	Duration (Days)
Requirement Analysis	7/3/2025	7/5/2025	3
Technical Documentation	7/6/2025	7/9/2025	4
Design Thinking Workshop	7/10/2025	7/11/2025	2
User App UI Design	7/16/2025	7/20/2025	5
Support Panel Implementation	8/25/2025	8/29/2025	5
Final Deployment	10/16/2025	10/18/2025	3

3.5 AIM AND OBJECTIVES OF THE PROJECT

The primary aim is to establish a secure, high-performance marketplace that simplifies the travel booking lifecycle. The technical methodology involves utilizing a decoupled microservices

architecture to ensure high availability and horizontal scalability, addressing complex hurdles such as race conditions during simultaneous booking attempts and maintaining state consistency across distributed services through asynchronous messaging.

Detailed Objectives and Methodology:

- **Full-Stack Implementation:** Develop a cross-platform mobile application using Flutter for the presentation layer, integrated with a Next.js web-based vendor dashboard. The methodology employs a shared API Gateway to route requests to specialized Django REST microservices, each containerized via Docker for environmental consistency.
- **Secure Authentication System:** Implement an OAuth2-compliant system utilizing JSON Web Tokens (JWT). The methodology includes a separate Identity Service to manage user credentials and issue short-lived access tokens alongside long-lived, rotating refresh tokens, ensuring a secure yet seamless session management experience.
- **Concurrency-Safe Booking:** Architect a reservation engine that eliminates double-booking errors. The methodology uses Redis-based distributed locking mechanisms to provide atomicity during the seat selection and payment verification phases, preventing over-allocation of travel packages during peak traffic.
- **Event-Driven Communication:** Build a resilient messaging pipeline using Apache Kafka. By employing the Transactional Outbox Pattern, the system ensures that database updates and event publishing happen atomically, facilitating reliable asynchronous updates for review systems and customer notifications.
- **Adherence to Clean Architecture:** Structure the Flutter application into clear Domain, Data, and Presentation layers. The methodology leverages Riverpod for reactive state management to isolate business logic from UI components, enhancing testability and long-term maintainability of the mobile client.

The project reached completion by late August 2025 for initial phases.

4. LITERATURE SURVEY

4.1 Mobile Frameworks

Flutter 3.x was selected as the primary frontend framework due to its Skia/Impeller rendering engines, which offer consistent performance across diverse hardware. By utilizing a single Dart codebase, the team minimized development overhead while maintaining the "look and feel" of native applications. This choice was critical for implementing the complex animations required for high-end travel discovery interfaces.

4.2 State Management and Routing

For robust state management, Riverpod provides a proactive approach to reactivity, preventing common "provider-not-found" errors through compile-time safety. Navigation is handled by GoRouter, which integrates seamlessly with Flutter's Router API to provide deep-linking capabilities—a vital feature for marketing travel packages via social media or email campaigns.

4.3 UI Standards

The design language adheres to Material Design 3 (M3), utilizing dynamic color schemes and adaptive layouts. Emphasis was placed on accessibility and intuitive navigation, ensuring that the package discovery process is accessible to users with varying levels of technical proficiency.

4.4 Microservices and Concurrency

The transition from monolithic to microservices architecture is driven by the need for fault isolation and independent scalability. Django REST Framework was selected for the backend services due to its mature ORM and security features. However, distributed systems introduce data consistency challenges. As per Kleppmann (2017), ensuring reliability requires robust messaging. Apache Kafka serves as the backbone for event streaming, while Redis Distributed Locks (Redlock) provide the necessary synchronization for critical sections, such as inventory management, where traditional ACID transactions across multiple databases are not feasible.

4.5 Event-Driven Architecture (EDA)

Planymo utilizes Apache Kafka as its event streaming backbone, enabling high-throughput and low-latency communication between services. Kafka is configured with multiple partitions per topic to ensure parallel processing and high availability. This decoupled architecture allows for

asynchronous data propagation, where events such as booking confirmations are broadcast to the notification and analytics engines without blocking the primary transaction flow.

4.6 Distributed Transaction Patterns

To maintain consistency across distributed databases, the system employs the Transactional Outbox Pattern and Saga choreography. The Outbox Pattern ensures atomicity by writing events to a local table within the same database transaction as the business entity. Saga choreography then manages long-lived transactions through a sequence of local transactions, where each service publishes events that trigger the next step or a compensating transaction in case of failure.

5. SYSTEM DESIGN

5.1 Architecture Overview

Planymo's architecture is built on a distributed microservices model, where each service owns its own database to ensure domain isolation. The system is split into three primary layers: the presentation layer (Flutter and Next.js), the API gateway layer (for request routing and rate limiting), and the service layer (Django microservices). This decoupling allows the discovery engine to scale independently from the payment and booking services.

1. **Client:** Flutter (Mobile) and Next.js (Web).
2. **Application:** Django REST microservices.
3. **Data:** PostgreSQL for persistence, Redis for distributed locking, and Kafka for event streaming.

5.2 Transactional Outbox Pattern

To handle the challenge of dual-writes in a microservices environment, the Outbox Pattern was implemented. When a user completes a booking, the transaction includes both the booking record and a message in the "outbox" table. A separate relay service then publishes this message to Kafka, ensuring that downstream services (like loyalty points or notifications) are eventually consistent without requiring heavy distributed locks.

5.3 Data Flow and Service Isolation

Planymo enforces strict database-per-service patterns to prevent tight coupling between domains. Inter-service communication is primarily asynchronous, mediated by Kafka topics. For instance, when the 'Booking Service' updates a reservation status, it publishes an event that the 'Notification Service' and 'Loyalty Service' consume independently. Synchronous communication via REST is limited to the API Gateway level to minimize latency and prevent cascading failures across the ecosystem.

5.4 High-Level Entity Relationship Diagram (ERD)

The system architecture features six independent PostgreSQL databases, each corresponding to a specific microservice (Identity, Package, Booking, Review, Payment, and Support). The ERD highlights a shared-nothing approach where data consistency is maintained via foreign-key-like logic at the application level rather than the database level, ensuring strict service isolation and allowing each schema to evolve independently.

5.5 Non-Functional Requirements (NFRs)

Key performance goals include sub-200ms response times for discovery queries and 99.9% availability for the booking engine. Security is enforced through TLS encryption in transit and AES-256 for sensitive data at rest. Scalability is achieved by containerizing services, allowing the system to handle sudden spikes in travel demand through horizontal pod autoscaling within the Kubernetes cluster.

6. MODULE DESCRIPTION

Module 1: Authentication & Identity

This module manages user lifecycle and security. Utilizing the specialized Identity microservice, it handles registration, login, and token rotation. In Flutter, the *authProvider* (Riverpod) watches the authentication state, automatically triggering redirects via GoRouter when tokens expire. It implements PKCE (Proof Key for Code Exchange) to secure mobile-specific authorization flows.

Module 2: Discovery & Search

The Discovery module leverages Elasticsearch integrated with the Django 'Package Service' to provide sub-second search results across thousands of listings. The Flutter UI uses *FutureProvider* to fetch paginated data, implementing "infinite scroll" and complex filtering (price range, dates, and ratings) with localized state management for the filter parameters.

Module 3: Booking & Reservation

The core of the platform, this module coordinates between the Booking Service and Redis. It implements a two-phase commit-like behavior where a "Pending" booking holds a Redis lock. If the payment succeeds, the lock is released and the database is updated; otherwise, the lock expires. Riverpod's *StateNotifier* manages the complex UI states: selecting, holding, paying, and confirmed.

Module 4: Vendor Management

The Vendor module provides the Next.js dashboard for tour operators to list packages and manage inventory. It interacts with the 'Inventory Service' via protected API endpoints. Key features include bulk availability updates, media uploads for package galleries, and real-time sales analytics visualized through Recharts.

Module 5: Booking-Verified Reviews

To ensure integrity, the Review Service only accepts submissions for 'Booking IDs' marked as 'Completed'. This cross-service validation happens via Kafka events. In the mobile app, the *reviewProvider* ensures that users only see the "Write a Review" button if the local state reflects a completed trip, verified against the backend.

Module 6: Support & Analytics

This module tracks system health and user issues. Support tickets are managed in a dedicated dashboard with WebSocket integration for live updates. The Flutter app includes a 'Help Center' component that pulls frequently asked questions and allows users to track their ticket status through a real-time sync mechanism.

Module 3: Payment Integration

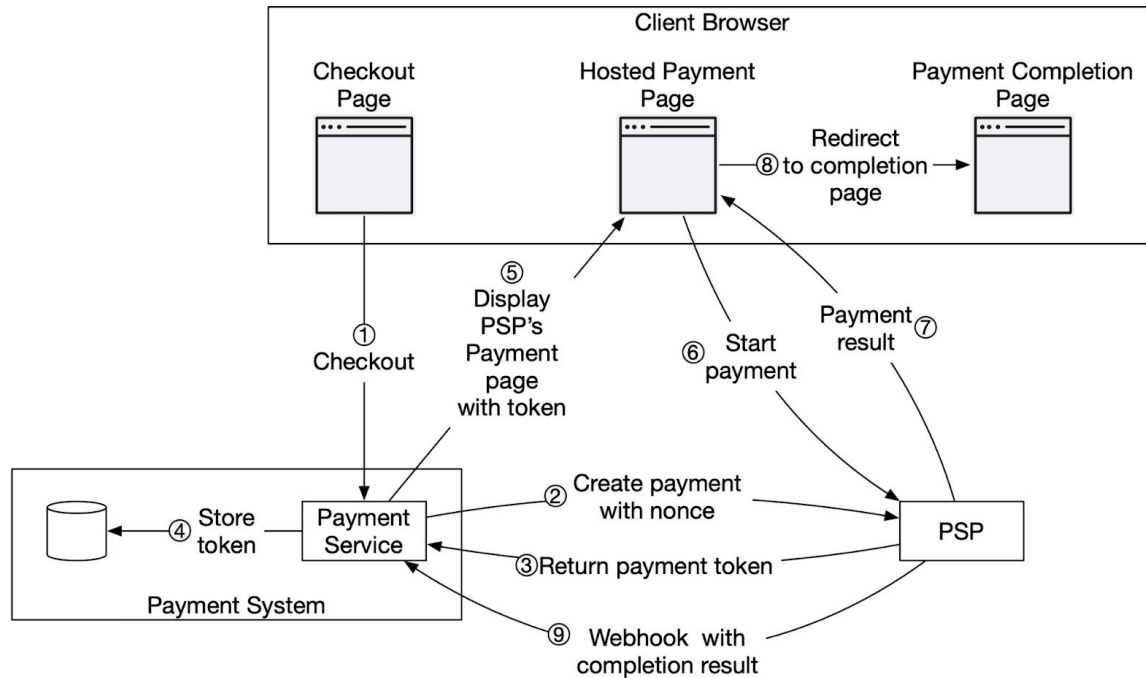
Payments are processed via the Razorpay API, supporting various methods including UPI, cards, and net banking. Our implementation uses a webhook-first approach, where the system only confirms a booking after receiving an asynchronous "payment.captured" event from the provider. This prevents "phantom bookings" that could occur if a user's connection drops after a successful payment but before the app can notify our server.

6.5 KEY BENEFITS OF THE PROPOSED SYSTEM

- **Unified Marketplace:** Eliminates fragmentation by consolidating discovery, booking, and reviews from multiple agencies into one mobile platform.
- **Verified Trust:** Ensures review credibility by only allowing submissions from users who have completed a booking, guaranteeing a Booking-Verified Review system.
- **Scalability & Reliability:** Leverages a microservices architecture with domain isolation and event streaming (Kafka) for independent scaling and fault tolerance.
- **Concurrency Safety:** Distributed booking locks (Redis) guarantee accurate seat/room availability management and prevent overselling, even under high simultaneous load.
- **Developer Experience:** Flutter's single codebase provides a native-quality experience on both Android and iOS, reducing development and maintenance effort.

7. ARCHITECTURE DIAGRAM

The Planymo architecture follows a layered distributed pattern. The Presentation layer (Flutter/Next.js) communicates with an NGINX API Gateway. Behind the gateway, specialized microservices (Auth, Booking, Search, Payment) interact with their respective PostgreSQL instances. A central Kafka cluster handles event distribution, while Redis acts as the caching and locking layer. The infrastructure is orchestrated using Docker Compose for local development and Kubernetes for production.



8. USE CASE DIAGRAM

The system identifies three primary actors: Customer, Vendor, and Support Administrator. Customers engage in searching, booking, and reviewing packages. Vendors manage package listings and view sales reports. Administrators oversee system health and resolve support tickets. Use cases include 'Secure Login', 'Search Package', 'Reserve Seat', 'Process Refund', and 'Moderate Review', with 'Reserve Seat' including an 'extend' relationship to 'Process Payment'.

9. HARDWARE SPECIFICATIONS

Development requires machines with at least 16GB RAM and Hexa-core processors to run microservice clusters. The production environment is hosted on AWS, utilizing EC2 t3.medium instances for individual services, RDS for managed PostgreSQL, and ElastiCache for Redis. Mobile testing covers devices with ARMv8 architecture and 4GB+ RAM.

10. SOFTWARE SPECIFICATIONS

Frontend: Flutter SDK 3.22+, Dart 3.4+, Next.js 14+. Backend: Python 3.11+, Django 4.2+, PostgreSQL 15, Redis 7. Messaging: Apache Kafka 3.6. DevOps: Docker 24+, Kubernetes 1.28. Testing: PyTest for backend, Flutter Test for mobile, and Playwright for web E2E testing.

11. TESTING AND VALIDATION

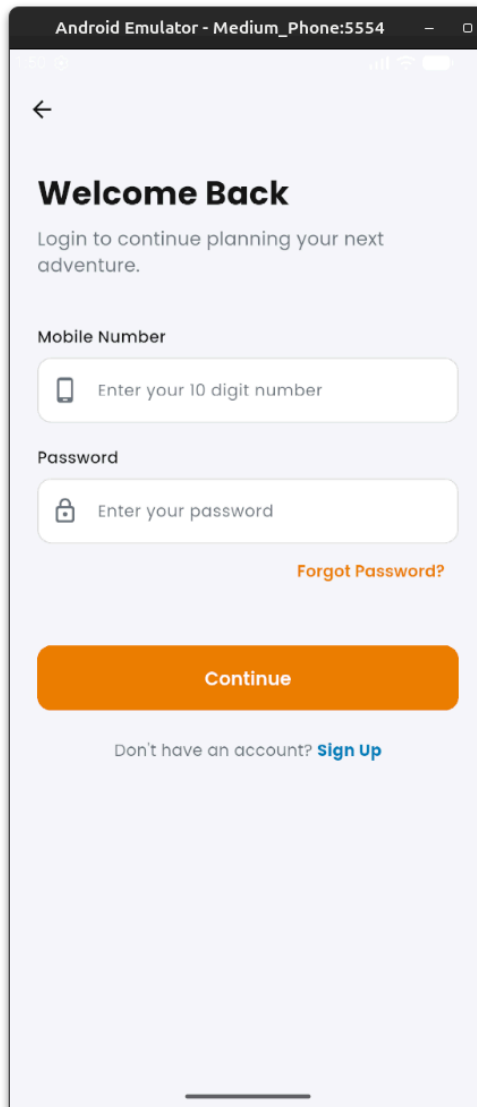
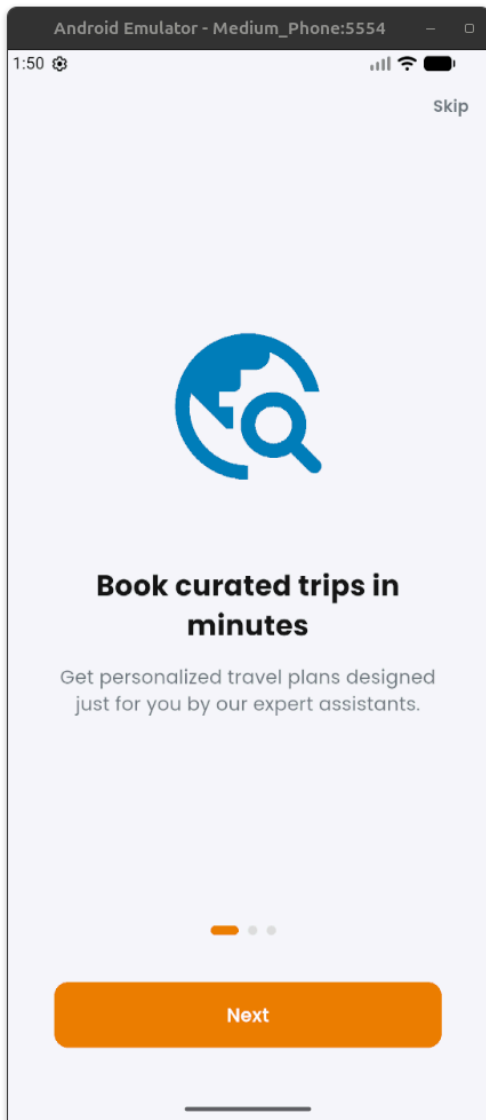
Planymo employs a multi-tier testing strategy. Unit tests ensure individual business logic correctness (e.g., price calculations). Integration tests validate inter-service Kafka message delivery. Concurrency testing is performed using JMeter to simulate 500+ simultaneous booking requests to verify Redis lock efficiency. End-to-end (E2E) tests automate the entire user journey from login to booking confirmation on both mobile and web platforms.

12. SECURITY ARCHITECTURE AND RISK ASSESSMENT

Planymo's security posture is built on a zero-trust model. Token management is handled by a centralized Identity Service issuing OAuth2/JWT tokens with PKCE for mobile clients. Security mechanisms include rate limiting at the NGINX gateway and automated vulnerability scanning in the CI/CD pipeline. The risk assessment identifies potential threats like distributed denial-of-service (DDoS) and credential stuffing, mitigating them through Web Application Firewalls (WAF) and multi-factor authentication requirements for high-privilege accounts.

13. DEPLOYMENT AND OPERATIONS

Planned upgrades include an in-app chat feature using WebSockets for direct traveler-to-vendor communication and an AI-powered recommendation engine utilizing collaborative filtering to suggest packages based on user browsing history. Additionally, multi-currency support and localized language packs are scheduled for the next major release phase.



Android Emulator - Medium_Phone:5554

←

Create Account

ⓘ Connection error or server unreachable

Full Name

 Enter your name

Mobile Number

 Enter 10 digit number

Password

 Min 8 characters, (A-Z, 0-9)

Confirm Password

 Re-enter your password

Continue

Already have an account? [Login](#)

15. CONCLUSION

In conclusion, Planymo demonstrates the successful application of modern software engineering principles—such as event-driven architecture and clean mobile design—to the travel industry. The resulting platform is not only scalable and performant but also provides a foundation for future AI-driven features. The project confirms that a microservices approach, while complex, provides the necessary reliability for high-stakes marketplace environments.

16. REFERENCES

1. Flutter Documentation. (2024). *Flutter — Build apps for any screen.*
2. World Travel & Tourism Council. (2025). *Economic Impact Research.*
3. Kleppmann, M. (2017). *Outbox Pattern in Microservices.*
4. Rousselet, R. (2021). *Riverpod Framework.*
5. Apache Kafka Documentation. (2024). *Event Streaming Platform.*
6. Planymo Project Documentation. (2025). *Feature Checklist and Support Logs.*